Reti Antonio

30232

Mini-StackOverflow
Project Lifecycle Documentation
Version 1.0

Revision History

Date	Version	Description	Author
19.04.2026	1.0	Project Specification & Elaboration – Iteration 1.1	Reti Antonio
10.05.2026	1.1	Elaboration – Iteration 1.2	Reti Antonio
26.05.2026	1.2	Elaboration – Iteration 2, Construction and Transition & Bibliography	Reti Antonio

1. Project Specification
2. Elaboration – Iteration 1.1
    2.1 Domain Model
    2.2 Architectural Design
        2.2.1 Conceptual Architecture
        2.2.2 Package Design
        2.2.3 Component Diagrams & Deployment Diagrams
3. Elaboration – Iteration 1.2
    3.1 Design Model
        3.1.2 Interaction View
        3.1.2 Class Design
    3.2 Data Model
    3.3 Unit Testing
4. Elaboration – Iteration 2
    4.1 Architectural Design Refinement
    4.2 Design Model Refinement
5. Construction and Transition
    5.1 System Testing
    5.2 Future Improvements
6. Bibliography

1. Project Specification

Purpose and Objectives The **Mini-StackOverflow** system is a specialized Q&A platform designed to facilitate technical knowledge sharing through a structured, client-server architecture. The primary objective is to maintain a high-quality knowledge base by allowing users to contribute content while providing moderators with robust tools to manage community standards.

Problem Domain Overview The system addresses the challenge of information noise and inappropriate content in open forums. It provides a centralized database for persistence and a clear hierarchy of roles to ensure that technical discussions remain relevant and moderated.

High-Level Requirements:

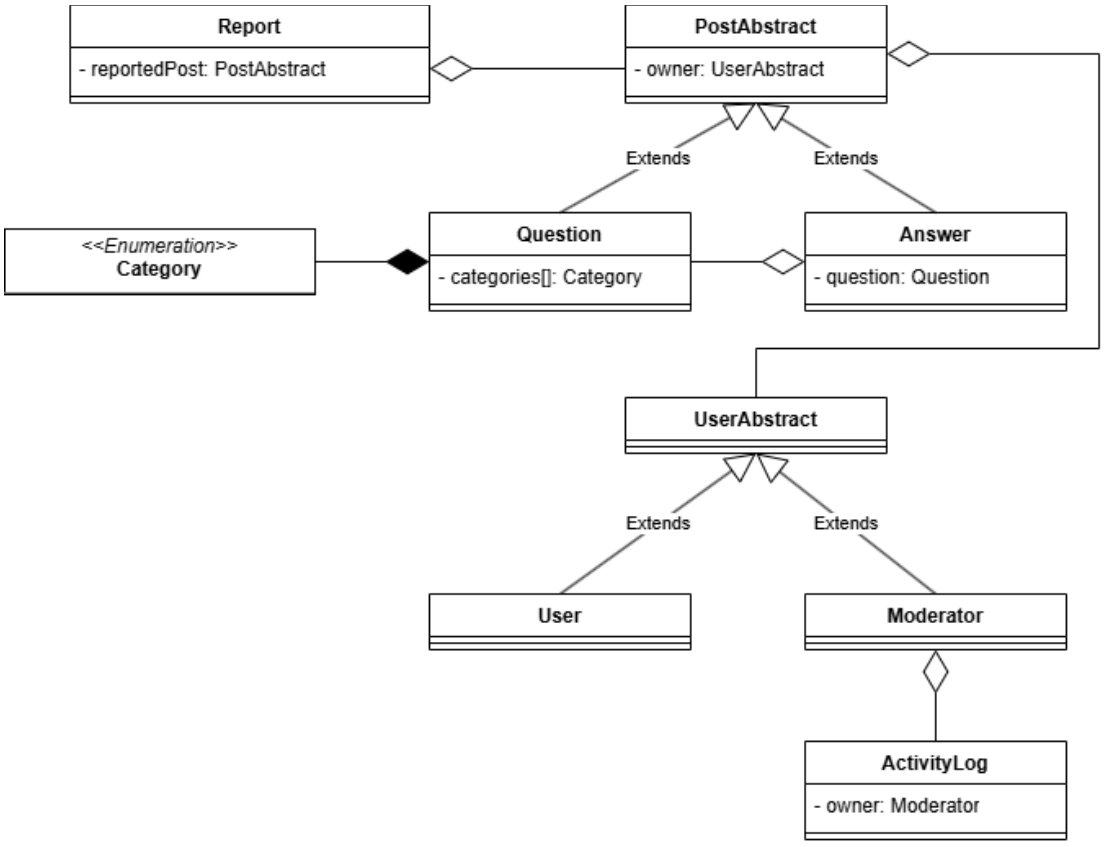
- **User Roles**: Support for Regular Users and privileged Moderators.
- **Post Management**: Creation of questions, posting of answers, and editing capabilities.
- **Moderation Workflow**: Reporting system for flagging inappropriate posts and administrative actions like user suspension.
- **System Intelligence**: Search functionality for existing questions and JSON-formatted activity logging.

2. Elaboration – Iteration 1.1

2.1 Domain Model

The domain model captures the core entities and their relationships within the problem domain.

- **User:** The central entity representing both Regular Users and Moderators.
- **Question:** A technical query created by a user containing tags and categories.
- **Answer:** A response created by a user linked specifically to a single question.
- **Report:** A dedicated model that links a User to a specific Post, carrying a "reason" and a "status" (Pending, Approved, Dismissed) to support the Approve flagged posts requirement.
- **Category (Enum):** A predefined set of technical tags (e.g., JAVA, PYTHON, SQL, C++) that ensures users categorize posts within a controlled taxonomy.
- **ActivityLog:** A persistent record of administrative actions taken by Moderators.



Model Class Diagram

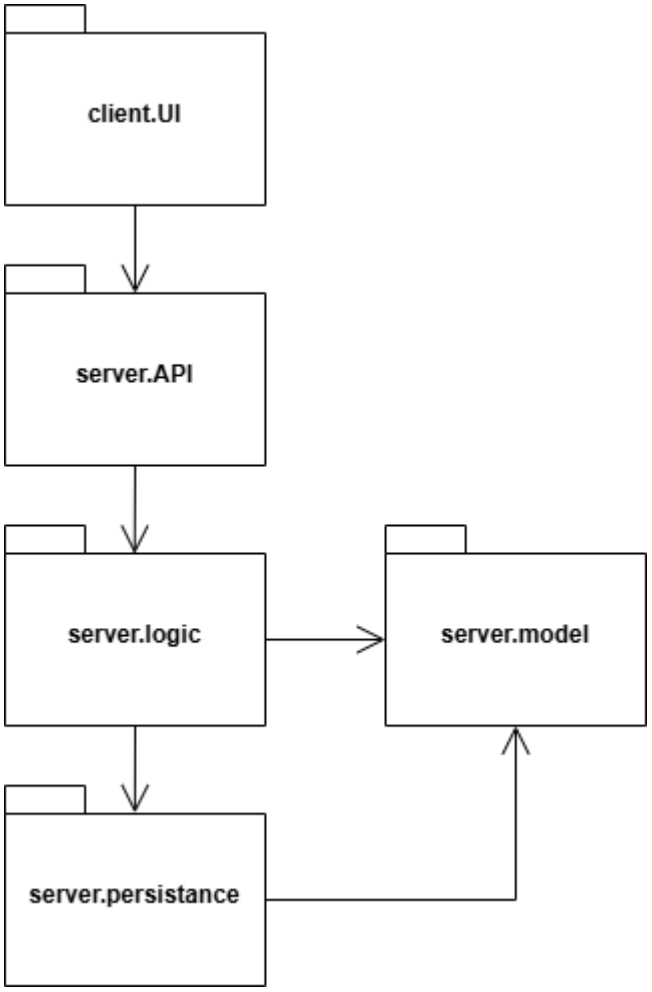
2.2 Architectural Design

2.2.1 Conceptual Architecture

The system utilizes a **Client-Server architectural style** paired with a **Layered Pattern**.

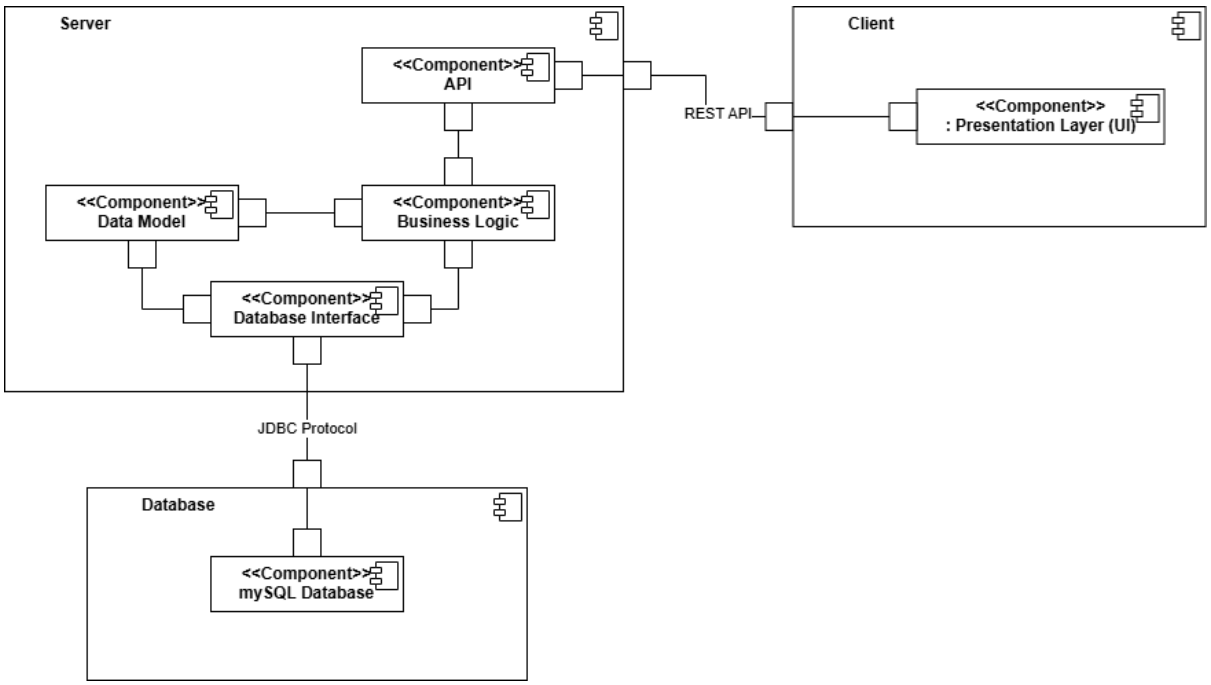
- **Motivation**: This style is chosen to support multiple concurrent users (clients) while maintaining data integrity in a centralized database (server).
- **Application**: The layers (Presentation, Business Logic, and Data Access) ensure that user interactions are decoupled from database operations, enhancing maintainability.

2.2.2 Package Design

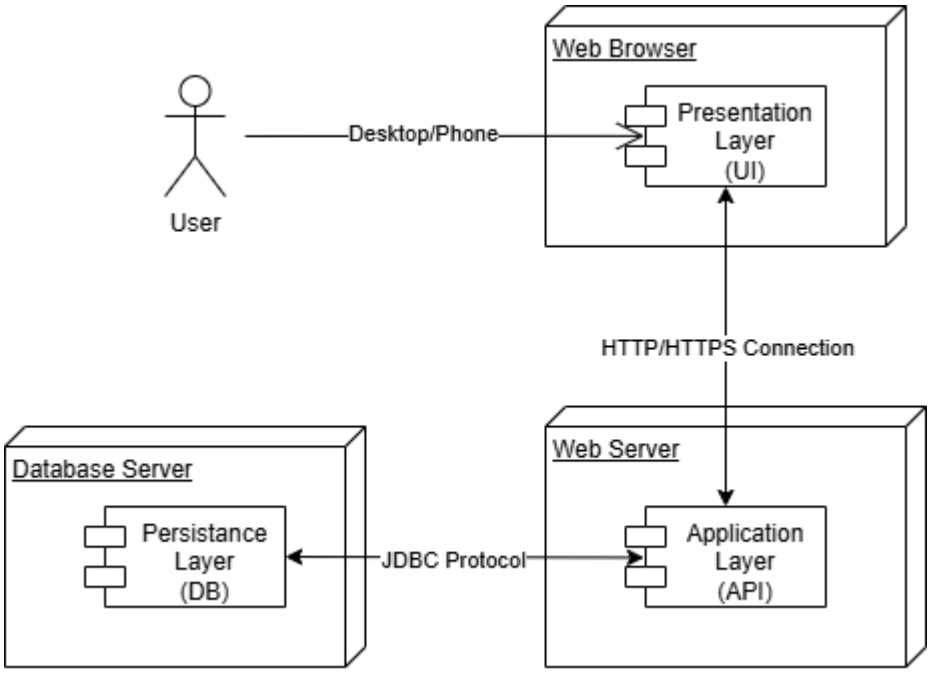


Package Diagram

2.2.3 Component Diagrams & Deployment Diagrams



Component Diagram



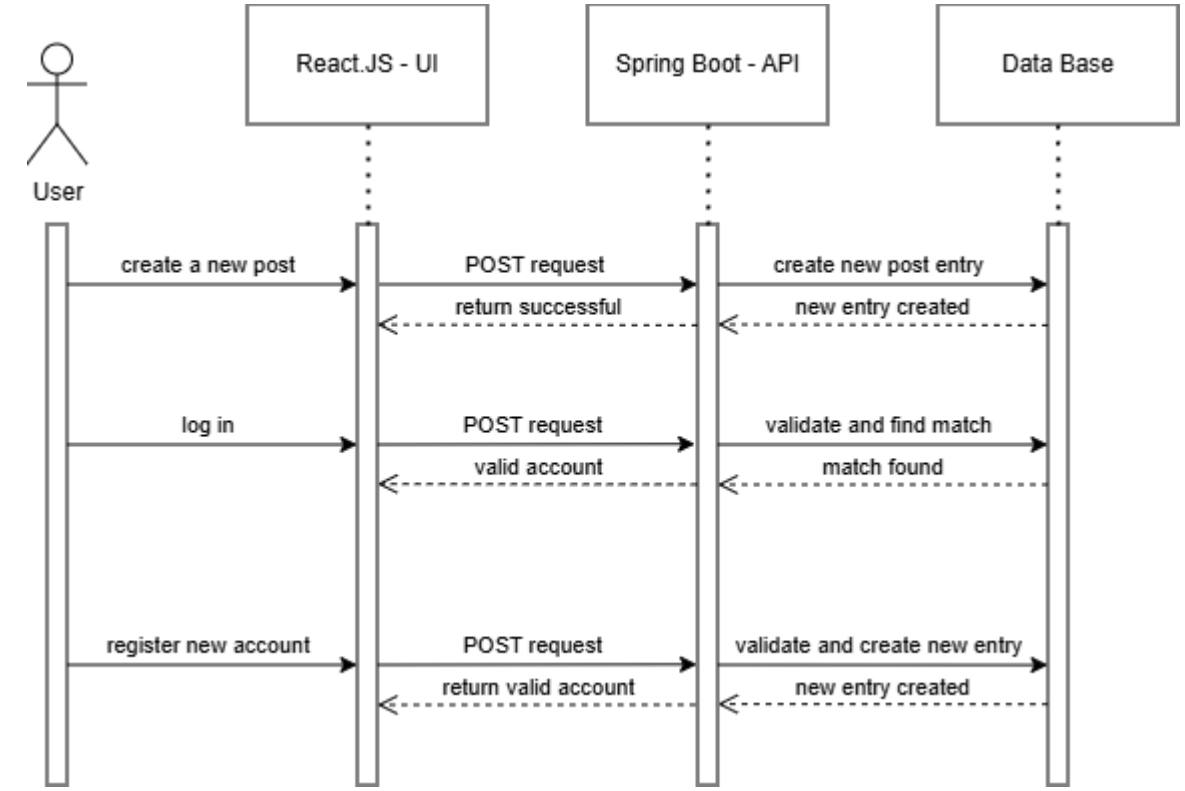
Deployment Diagram

3. Elaboration – Iteration 1.2

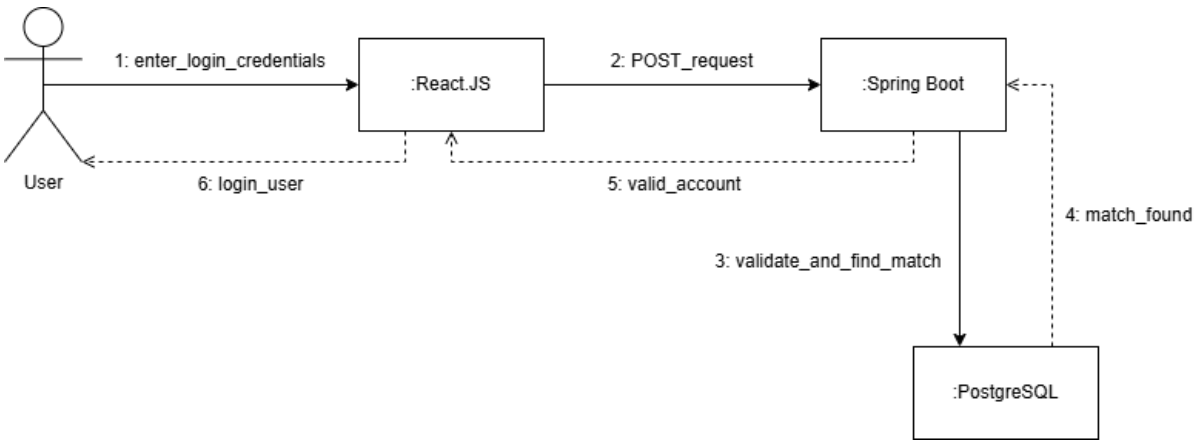
[Functional Baseline]

3.1 Design Model

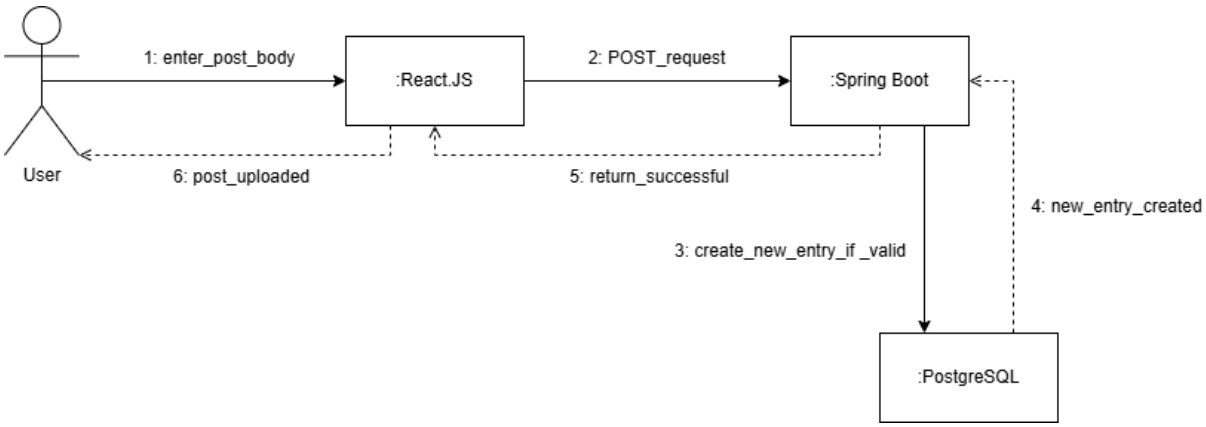
3.1.2 Interaction View



Sequence Diagram

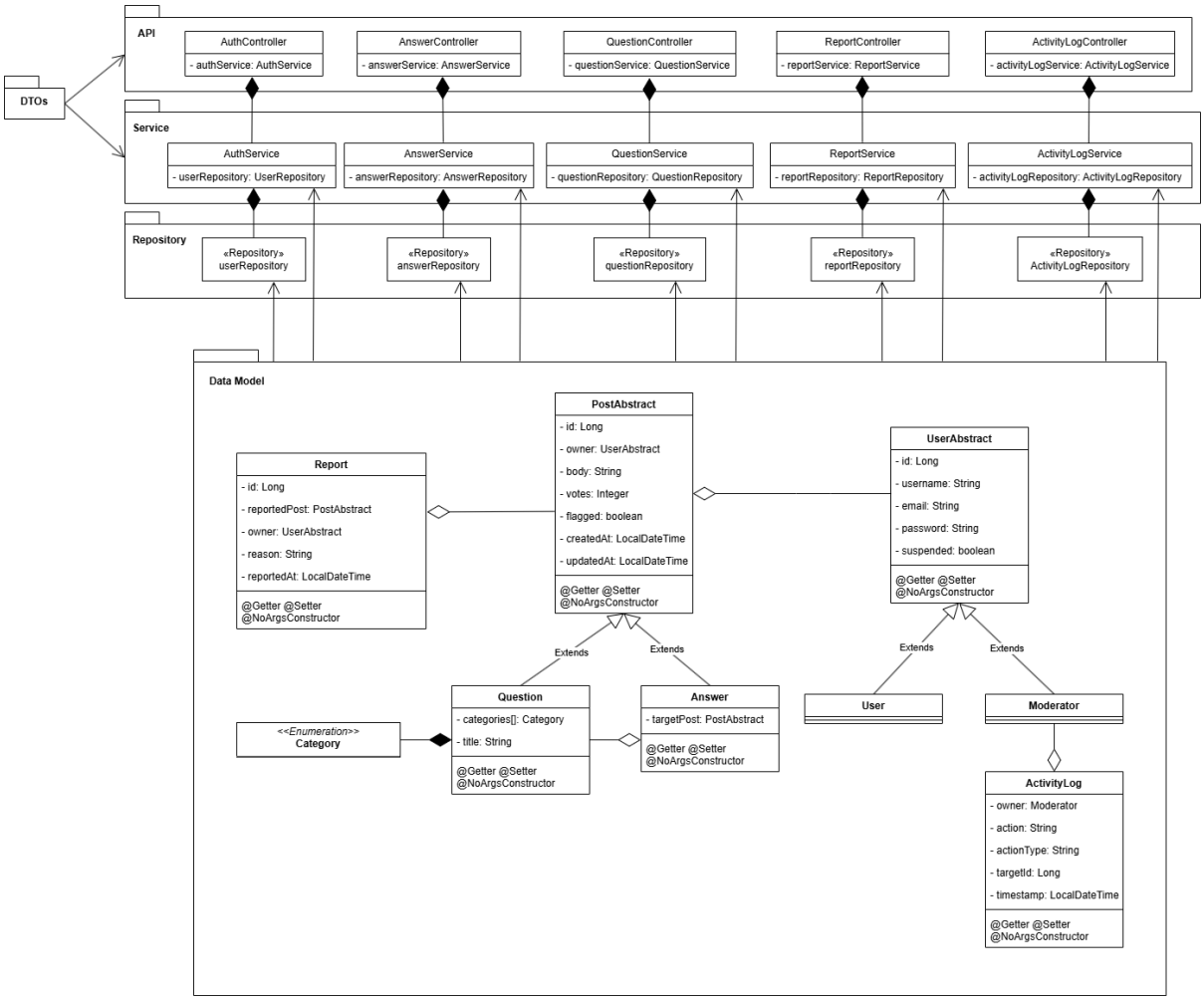


Login – Communication Diagram



Creating a Post – Communication Diagram

3.1.2 Class Design



3.2 Data Model



PostgreSQL - ER Diagram

3.3 Unit Testing

The unit testing strategy for the Mini-StackOverflow application focuses on validating the business logic layer in isolation, ensuring each service behaves correctly independently of the database, security context, or HTTP layer.

AuthService — Register with valid data — Returns token and correct username

AuthService — Register with duplicate username — Throws "Username already taken"

AuthService — Login with correct credentials — Returns valid JWT token

AuthService — Login with suspended account — Throws "Account is suspended"

QuestionService — Create question with valid data — Returns response with correct title and owner

QuestionService — Get question by non-existent ID — Throws "Question not found"

QuestionService — Update another user's question — Throws "You can only edit your own questions"

QuestionService — Vote up on a question — Returns response with votes incremented by 1

AnswerService — Post answer to existing question — Returns response linked to correct question

AnswerService — Update another user's answer — Throws "You can only edit your own answers"

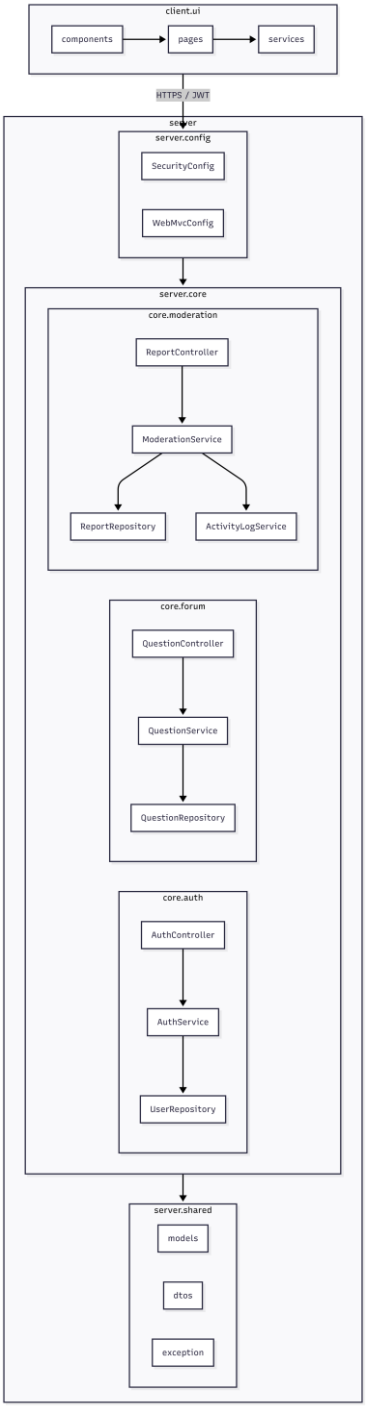
AnswerService — Vote down on an answer — Returns response with votes decremented by 1

JwtUtil — Validate a freshly generated token — isTokenValid returns true

JwtUtil — Validate a tampered token — isTokenValid returns false

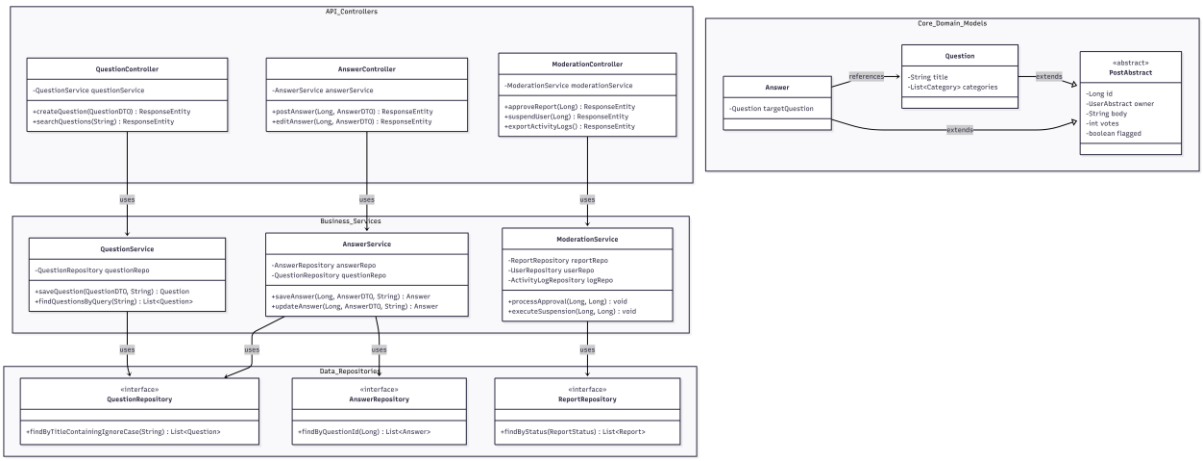
4. Elaboration – Iteration 2

4.1 Architectural Design Refinement



Packet diagram

4.2 Design Model Refinement



Class Diagram

5. Construction and Transition

5.1 System Testing

Integration and end-to-end system testing verified that the React.js frontend, Spring Boot API, and PostgreSQL database collaborate flawlessly under real-world scenarios. Testing used a staged integration approach.

Integration Test Scenarios:

- **Test Scenario 1:** Comprehensive Moderation Pipeline
  - **Method:** Programmatic simulation of an content violation.
  - **Execution:** A test account creates a post. A separate user issues a POST request to ReportController. The system checks the database, validates the status change to PENDING, and updates the moderator interface.
  - Expected Outcome: The post status flags correctly, and a row updates in the PostgreSQL reports table.
- **Test Scenario 2:** Disciplinary Scope Verification
  - **Method:** Security firewall validation.
  - **Execution:** A moderator issues a user suspension command via the client application. The target user immediately attempts a POST request to upload content.
  - **Expected Outcome:** The system acts on the modified state, rejects the entry, throws a 403 Forbidden error, and logs the moderator's action inside the database tracking system.

5.2 Future Improvements

To expand the scope of the Mini-StackOverflow application in future development lifecycle iterations, the following structural enhancements are proposed:

- **Full-Text Elastic Search Integration:** Replacing basic relational database queries with an indexed caching layer to decrease search latencies across millions of concurrent text bodies.
- **Automated Content Classification:** Utilizing light Machine Learning pipelines on the server backend to automatically parse incoming question texts and match them against the `Category` Enum taxonomy.
- **Real-Time Notification Gateways:** Implementing WebSocket protocols alongside standard HTTP to instantly alert users when an answer is uploaded to their pending question threads.

6. Bibliography

- **Figma Inc.** (2026). Figma Help Center: Design Systems and Prototyping Tools. Available at: https://help.figma.com/ (Accessed for the creation of the system's high-fidelity UI wireframes and user interface prototyping).
- **Meta Platforms Inc.** (2026). React Documentation: Managing Application State and Component Lifecycles. Available at: https://react.dev/ (Accessed for structuring the modular client frontend architecture and component hierarchies).
- **Broadcom Inc. / VMware**. (2026). Spring Boot Framework Documentation: Core Technologies and RESTful Web Services. Available at: https://spring.io/projects/spring-boot (Accessed for implementing the decoupled controller-service-repository patterns and REST API route handling).
- **Hibernate Community**. (2026). Hibernate ORM Core User Guide. Available at: https://hibernate.org/orm/documentation/ (Accessed for configuring Object-Relational Mapping (ORM) annotations, data repositories, and entity inheritance structures).